Custom Scripts for Life Sciences

Custom scripts are programmatic tools for data validation across Life Sciences Cloud for Customer Engagement on desktop and both online and offline in the Life Sciences Cloud mobile app. Custom scripts are used in Life Sciences workflow management and visit management.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Custom scripts execute during Life Sciences workflow actions to support more complex business scenarios than standard Salesforce validation rules. Unlike Salesforce validation rules, custom scripts consolidate validation logic and can query any accessible database data, rather than just related records. Custom scripts also offer more dynamic user feedback, providing warnings and custom error messages when users execute actions.

You can create these types of custom scripts.

- Checklist: Checklist custom scripts are used in Life Sciences Cloud for Customer Engagement workflows. Checklist scripts help users understand all of the steps to take before they can move to the next stage in the workflow. These scripts are executed when a user clicks the info icon on Record Update actions.
- Validation: Validation custom scripts are used in Life Sciences Cloud for Customer Engagement workflows. Validation scripts make sure that business rules are met before users can move a record to the next stage in a workflow. These scripts are executed when a user runs any workflow action.
- Visit Action Validation: Visit action validation custom scripts validate business rules before a user can sign and submit a visit. These scripts are executed when a user runs an action to sign or submit visits.

Understand the Format and Output for Life Sciences Custom Scripts

Understand the format and output for custom scripts in Life Sciences Cloud for Customer Engagement.

Best Practices for Life Sciences Custom Scripts

Follow these best practices when you create custom scripts for Life Sciences Cloud for Customer Engagement.

Test Custom Scripts for Life Sciences

Because custom scripts aren't typical Lightning web components, you can't test them in the same way. Write custom Jest tests to test the Lightning web components that you create as custom scripts for workflows in Life Sciences Cloud for Customer Engagement.

Troubleshoot Custom Scripts for Life Sciences

To troubleshoot and debug custom scripts for Life Sciences Cloud for Customer Engagement workflow validation, use Chrome Developer Tools. The Console tab shows log information, while the Network

tab helps monitor network requests, responses, and performance.

Life Sciences Custom Scripts Reference
Understand the available JavaScript classes, functions, and variables that you can include in a Lightning web component for custom scripts in Life Sciences Cloud for Customer Engagement.

Understand the Format and Output for Life Sciences Custom Scripts

Understand the format and output for custom scripts in Life Sciences Cloud for Customer Engagement.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Format

You create custom scripts as headless Lightning web components, meaning that the component includes only the JavaScript file and the metadata configuration file. See Create Lightning Web Components.

In the LWC, contain custom script logic in a self-calling function.

```
(() => {
    // Add custom script logic here.
    ...
})();
```

Output

Custom scripts return an array of JavaScript objects with two defined properties.

Property	Type	Details
title	String	The message that users see. This can be a custom label name that can be translated to match the current user's language.
status	String	The type of message. Statuses are:

Property	Type	Details
		• success: Shows a success message for checklist scripts. Not displayed for validation scripts. • warning: Shows a warning message for checklist scripts. For validation scripts, if there are warnings but not errors, users see all warnings in one window, and they can choose to continue. • error: Shows an error message for checklist scripts. Shows an error window for validation scripts.

See this example output.

```
return [
    {
        title: "Success Message",
        status: "success",
    },
    {
        title: "Success_Message_Custom_Label",
        status: "success",
    },
    {
        title: "Warning Message",
        status: "warning",
    },
    {
        title: "Warning_Message_Custom_Label",
        status: "success",
    },
    {
        title: "Error Message",
        status: "error",
    },
    {
        title: "Error_Message_Custom_Label",
```

```
        status: "success",
    }
];
```

Best Practices for Life Sciences Custom Scripts

Follow these best practices when you create custom scripts for Life Sciences Cloud for Customer Engagement.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Asynchronous Calls

Database queries are synchronous on the Life Sciences Cloud mobile app but asynchronous on desktop. To support asynchronous calls, enclose each validation in an `async` function.

```
(() => {
    async function inquiryQuestionsValidation() {
        try {
            let inquiryQuestions = await db.query(
                "InquiryQuestion",
                await new ConditionBuilder(
                    "InquiryQuestion",
                    new FieldCondition("InquiryId", "=", getRecordId(record))
                ).build(),
                ["Id", "Name"]
            );

            if (
                inquiryQuestions === null ||
                inquiryQuestions === undefined ||
                inquiryQuestions.length === 0
            ) {
                return {
                    title: "No Inquiry Questions Found",
                    status: "error",
                };
```

```
            }

            return {
                title: "Inquiry Questions Added",
                status: "success",
            }

        } catch(error) {
            return {
                title: "Caught Exception During Inquiry Questions Validation",
                status: "error"
            }
        }
    }

    function getRecordId(record) {
        let recordId = record.stringValue("Id");
        return recordId ? recordId : record.stringValue("uid");
    }

    return [inquiryQuestionsValidation()];
})();
```

Database Queries

- To improve performance, always specify fields in database queries.
- Rather than making multiple WHERE clause queries to the database, filter queries in the JavaScript instead.

Error Handling

To make sure that you “catch” JavaScript errors, use a `Try-Catch` block in all validation functions in custom scripts. Otherwise, when JavaScript errors occur, validation rules don’t show in the UI.

If `enableAccessErrors()` is called before executing a function to retrieve field values, the function returns a JavaScript error for the inaccessible field. If an error occurs but isn’t caught by a `Try-Catch` block:

- On desktop, users see only a field access error for checklist and validation scripts.
- On the Life Sciences Cloud mobile app, users see a field access error for checklist scripts, but validation scripts fail.

Testing

Because errors can sometimes occur in only one environment, we recommend testing custom scripts on both desktop and in the Life Sciences Cloud mobile app.

Test Custom Scripts for Life Sciences

Because custom scripts aren't typical Lightning web components, you can't test them in the same way. Write custom Jest tests to test the Lightning web components that you create as custom scripts for workflows in Life Sciences Cloud for Customer Engagement.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

See Test Lightning Web Components.

1. Load the script.
   a. Read the script's contents from the file system.
   b. Create a new function that accepts all necessary parameters for the script and includes the script's content as the final parameter.
   The parameters that you pass into the function should be mocked versions of the classes and variables that are used in the script. The last parameter should be `return`, followed by the content of your script file.

```
// Load the script
const scriptContent = fs.readFileSync(path.resolve(__dirname, '../inquiryQ
uestionsValidation.js'));
const scriptFunction = new Function('env', 'record', 'db', 'ConditionBuild
er', 'FieldCondition', `return ${scriptContent.toString()}`);
```

2. Mock custom script classes and variables.
   To create custom script tests, you must create mocked versions of any out-of-the-box classes and variables and pass them in during testing. If you don't pass in mocked classes and variables, test failures occur.

```
// Mock the classes and variables that are used in the script
const mockEnv = {
    getOption: jest.fn(),
};
const mockDb = {
    query: jest.fn(),
```

```
};
const mockRecord = {
    stringValue: jest.fn(),
};
const mockConditionBuilder = jest.fn().mockImplementation(() => ({
    build: jest.fn().mockResolvedValue({}),
}));
const mockFieldCondition = jest.fn();
```

3. Set up a test using this structure.
   a. Mock the data that's necessary for your test scenario.
   b. Invoke your script's function, and provide mocked classes and variables.
   c. To make sure that the complete result is available, allow all validation functions to resolve.
   d. Verify the results.

```
// Mock the db.query method to return the desired value
mockDb.query.mockResolvedValue([{ Id: '1', Name: 'Question 1' }]);

// The scriptFunction returns an array containing a Promise
const promiseArray = scriptFunction(mockEnv, mockRecord, mockDb, mockConditi
onBuilder, mockFieldCondition);

// Wait for the Promise in the array to resolve
const result = await Promise.all(promiseArray);

// Assert the result is as expected
expect(result).toEqual([{
    title: 'Inquiry Questions Added',
    status: 'success',
}]);
```

Example Test Suite for Life Sciences Custom Scripts
This example provides a test suite for a custom script in Life Sciences Cloud for Customer Engagement.

Example Test Suite for Life Sciences Custom Scripts

This example provides a test suite for a custom script in Life Sciences Cloud for Customer Engagement.

This example Jest test suite is designed to test the example custom script in Best Practices for Life Sciences Custom Scripts.

```
import fs from 'fs';
import path from 'path';
```

```
// Load the script
const scriptContent = fs.readFileSync(path.resolve(__dirname, '../inquiryQuest
ionsValidation.js'));
const scriptFunction = new Function('env', 'record', 'db', 'ConditionBuilder',
'FieldCondition', `return ${scriptContent.toString()}`);

// Mock the classes and variables that are used in the script
const mockEnv = {
    getOption: jest.fn(),
};
const mockDb = {
    query: jest.fn(),
};
const mockRecord = {
    stringValue: jest.fn(),
};
const mockConditionBuilder = jest.fn().mockImplementation(() => ({
    build: jest.fn().mockResolvedValue({}),
}));
const mockFieldCondition = jest.fn();


describe('inquiryQuestionsValidation', () => {

    beforeEach(() => {
        // Clear all mocks and reset modules to ensure a clean state
        jest.clearAllMocks();
    });

    test('should return Inquiry Questions Added when there are inquiry questio
ns', async () => {
        // Mock the db.query method to return the desired value
        mockDb.query.mockResolvedValue([{ Id: '1', Name: 'Question 1' }]);

        // The scriptFunction returns an array containing a Promise
        const promiseArray = scriptFunction(mockEnv, mockRecord, mockDb, mockC
onditionBuilder, mockFieldCondition);

        // Wait for the Promise in the array to resolve
        const result = await Promise.all(promiseArray);

        // Assert that the result is as expected
        expect(result).toEqual([{
```

```
            title: 'Inquiry Questions Added',
            status: 'success',
        }]);
    });

    test('should return No Inquiry Questions Found when there are no inquiry q
uestions', async () => {
        // Mock the db.query method to return the desired value
        mockDb.query.mockResolvedValue([]);

        // The scriptFunction returns an array containing a Promise
        const promiseArray = scriptFunction(mockEnv, mockRecord, mockDb, mockC
onditionBuilder, mockFieldCondition);

        // Wait for the Promise in the array to resolve
        const result = await Promise.all(promiseArray);

        // Assert that the result is as expected
        expect(result).toEqual([{
            title: 'No Inquiry Questions Found',
            status: 'error',
        }]);
    });

    test('should return Caught Exception During Inquiry Questions Validation w
hen there is an error', async () => {
        // Mock the db.query method to return the desired value
        mockDb.query.mockRejectedValue(new Error('Error'));

        // The scriptFunction returns an array containing a Promise
        const promiseArray = scriptFunction(mockEnv, mockRecord, mockDb, mockC
onditionBuilder, mockFieldCondition);

        // Wait for the Promise in the array to resolve
        const result = await Promise.all(promiseArray);

        // Assert that the result is as expected
        expect(result).toEqual([{
            title: 'Caught Exception During Inquiry Questions Validation',
            status: 'error',
        }]);
    });

});
```

Troubleshoot Custom Scripts for Life Sciences

To troubleshoot and debug custom scripts for Life Sciences Cloud for Customer Engagement workflow validation, use Chrome Developer Tools. The Console tab shows log information, while the Network tab helps monitor network requests, responses, and performance.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Troubleshoot in the Console Tab

Troubleshoot custom scripts from the Chrome Developer Tools Console tab.

1. Open Chrome Developer Tools.
2. Go to the Console tab.
3. Review the output for custom scripts.
   Custom scripts can use these messages to provide insights into script execution, variable values, and debugging information.
   - `console.log()`
   - `console.warn()`
   - `console.error()`
4. To investigate specific issues, filter the logs by type.
   For example, filter by Error, Warning, or Info.

Troubleshoot Network Requests and Responses

Troubleshoot custom scripts from the Chrome Developer Tools Networks tab. When a custom script makes asynchronous calls, you can monitor these requests in the Network tab.

1. Open Chrome Developer Tools.
2. Go to the Network tab.
3. To monitor network activity, look for requests that are related to your custom script and how it queries or retrieves data.
4. To inspect requests and responses, click a specific network request and review its details, including the headers, payload, and response.
5. To show data that's returned by the server, select the Response tab from within a network request.
   The data that's returned by the server can help you debug issues that are related to data retrieval.

Life Sciences Custom Scripts Reference

Understand the available JavaScript classes, functions, and variables that you can include in a Lightning web component for custom scripts in Life Sciences Cloud for Customer Engagement.

REQUIRED EDITIONS

Available in: Lightning Experience

Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Classes for Life Sciences Custom Scripts

Life Sciences Cloud for Customer Engagement supports various JavaScript classes in custom scripts. These classes enable you to create powerful and flexible logic within the script, particularly for constructing queries and managing data. Each class includes a description of its purpose, constructor details, parameters, methods, and examples.

REQUIRED EDITIONS

Available in: Lightning Experience

Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

An abstract class that serves as the foundation for all conditions and lacks a concrete implementation.

ConditionBuilder Class
Builds a WHERE clause to use as part of a SOQL query. Validates if the current user has access to the fields that are used in conditions.

ConditionEnhancedBuilder Class
Serves the same purpose as the ConditionBuilder class. Only use the ConditionEnhancedBuilder class with the db.bulkQuery function. The ConditionEnhancedBuilder class doesn't validate whether the current user has access to the fields that are used in the query conditions.

DateFieldCondition Class
Builds a SOQL condition for a date field based on the specified operator. Extends FieldCondition.

DateTimeFieldCondition Class
Builds a SOQL condition for a datetime field based on the specified operator. Extends FieldCondition.

FieldCondition Class
Builds a SOQL condition for a field based on the specified operator. Extends OperatorCondition.

GroupCondition Class
Builds a SOQL condition based on multiple nested conditions. Extends Condition.

OperatorCondition Class
Extends the Condition class. Used as a super class for specific conditions.

OrCondition Class
Builds a SOQL condition based on multiple nested conditions. Extends GroupCondition.

Query Class
Creates a SOQL query based on an input. Only use the Query class as a subquery of SetCondition. To build a complex WHERE clause, pass SetCondition to ConditionBuilder.

SetCondition Class
Builds a SOQL condition for a field whose value is in a specified range. Extends OperatorCondition.

AndCondition Class
AndCondition Class

Builds a SOQL condition based on multiple nested conditions. Extends GroupCondition.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor()

Builds a GroupCondition with **AND** as the nesting operator.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
    "HealthcareProvider",
    new AndCondition()
        .add(new FieldCondition("Status", "=", "Active"))
        .add(new FieldCondition("IsPrimaryProvider", "=", true))
).build()

// Returns: "(Status = \'Active\') AND (IsPrimaryProvider = true)"
```

Condition Class
Condition Class

An abstract class that serves as the foundation for all conditions and lacks a concrete implementation.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Important Don't use the Condition class directly. Instead, use the predefined classes that extend the Condition class. Provide an instance of a Condition subclass as a parameter for the ConditionBuilder. The ConditionBuilder then uses the Condition subclass to construct the WHERE clause part of the query.

getRequiredFields()

Returns an empty object.

toSoql()

Returns an empty string.

ConditionBuilder Class
ConditionBuilder Class

Builds a WHERE clause to use as part of a SOQL query. Validates if the current user has access to the fields that are used in conditions.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(objectType, condition)

Creates an instance of ConditionBuilder.

This method accepts these parameters.

Parameter	Type	Description
objectType	String	The object for which the query is built.
condition	Condition	The condition to use for the query.

build()

Builds the condition based on the provided object and condition. Checks if the user has access to the fields that are used in conditions, and returns an exception if they don't have access.

Example

```
await new ConditionBuilder(
   "Account",
   new FieldCondition("Name", "=", "sForce")
```

```
).build()

// Returns: "Name = 'sForce'"
```

ConditionEnhancedBuilder Class ConditionEnhancedBuilder Class

Serves the same purpose as the ConditionBuilder class. Only use the ConditionEnhancedBuilder class with the db.bulkQuery function. The ConditionEnhancedBuilder class doesn't validate whether the current user has access to the fields that are used in the query conditions.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(objectType, condition)

Creates an instance of ConditionEnhancedBuilder.

This method accepts these parameters.

Parameter	Type	Description
objectType	String	The object for which the query is built.
condition	Condition	The condition to use for the query.

build()

Returns a JavaScript object with a SOQL string and a map of fields that are listed in the condition for each JavaScript object.

Example

```
await new ConditionEnhancedBuilder(
   "Account",
```

```
   new FieldCondition("Name", "=", "sForce")
).build()

/**
 * Returns this JavaScript object:
 * {
 *     soqlString: "Name = 'sForce'",
 *     fieldNamesByObjectNames: {"Account": ["Name"]}
 * }
 */
```

DateFieldCondition Class
DateFieldCondition Class

Builds a SOQL condition for a date field based on the specified operator. Extends FieldCondition.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(dateField, operator, dateValue)

Creates an instance of DateFieldCondition.

This method accepts these parameters.

Parameter	Type	Description
dateField	String	The name of the field to use as part of a condition.
operator	String	The operator to use as part of a condition.
dateValue	Date	The date to use as part of a condition.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
    "HealthcareProvider",
    new DateFieldCondition("InitialStartDate", "=", new Date())
).build()

// Returns: "InitialStartDate = 2024-08-12"
```

DateTimeFieldCondition Class
DateTimeFieldCondition Class

Builds a SOQL condition for a datetime field based on the specified operator. Extends FieldCondition.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(dateTimeField, operator, dateTimeValue)

Creates an instance of DateTimeFieldCondition.

This method accepts these parameters.

Parameter	Type	Description
dateTimeField	String	The name of the field to use as part of a condition.
operator	String	The operator to use as part of a condition.
dateTimeValue	Date	The date to use as part of a condition.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
    "Inquiry",
    new DateTimeFieldCondition("CreatedDate", "=", new Date())
    ).build()

    // Returns: "CreatedDate = 2024-08-12T12:47:55.594Z"
```

FieldCondition Class
FieldCondition Class

Builds a SOQL condition for a field based on the specified operator. Extends OperatorCondition.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(field, operator, value)

Creates an instance of FieldCondition.

This method accepts these parameters.

Parameter	Type	Description
field	String	The name of the field to use as part of a condition.
operator	String	The operator to use as part of a condition.
value	String	The value to use as part of a condition. A null value is supported for the = and != operators.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
   "Inquiry",
   new FieldCondition("Type", "=", "MedicalInquiry")
).build()

// Returns: "Type = 'MedicalInquiry'"
```

GroupCondition Class
GroupCondition Class

Builds a SOQL condition based on multiple nested conditions. Extends Condition.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(nestingOperator)

Creates an instance of GroupCondition.

This method accepts these parameters.

Parameter	Type	Description
nestingOperator	String	The operator to use between nested conditions. Supported operators are **AND** and **OR**.

add(condition)

Adds a nested condition to the group condition.

This method accepts these parameters.

Parameter	Type	Description
condition	Condition	The condition to add to the group condition.

getRequiredFields(objectType)

Returns the required fields for a specified object type.

This method accepts these parameters.

Parameter	Type	Description
objectType	String	The name of the object to get the required fields for.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
    "HealthcareProvider",
    new GroupCondition("AND")
        .add(new FieldCondition("Status", "=", "Active"))
        .add(new FieldCondition("IsPrimaryProvider", "=", true))
).build()

// Returns: "(Status = \'Active\') AND (IsPrimaryProvider = true)"
```

OperatorCondition Class

OperatorCondition Class

Extends the Condition class. Used as a super class for specific conditions.

REQUIRED EDITIONS

Available in: Lightning Experience

Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(field, operator)

Creates an instance of OperatorCondition.

This method accepts these parameters.

Parameter	Type	Description
field	String	The name of the field to use as part of a condition.
operator	String	The operator to use as part of a condition.
value	Array or Query	An array of values to use as part of a condition, or an instance of a Query object that specifies the field values to query.

getRequiredFields(objectType)

Returns the required fields for a specified object type.

This method accepts these parameters.

Parameter	Type	Description
objectType	String	The name of the object to get the required fields for.

toSoql()

Returns the generated SOQL query as a string.

OrCondition Class
OrCondition Class

Builds a SOQL condition based on multiple nested conditions. Extends GroupCondition.

constructor()

Builds a GroupCondition with **OR** as the nesting operator.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
    "HealthcareProvider",
    new OrCondition()
        .add(new FieldCondition("Status", "=", "Active"))
        .add(new FieldCondition("IsPrimaryProvider", "=", true))
).build()

// Returns: "(Status = \'Active\') OR (IsPrimaryProvider = true)"
```

Query Class

Query Class

Creates a SOQL query based on an input. Only use the Query class as a subquery of SetCondition. To build a complex WHERE clause, pass SetCondition to ConditionBuilder.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor()

Creates an empty instance of Query.

select(field)

Specifies the field name to select. Only one field name can be selected. Returns an instance of Query.

This method accepts these parameters.

Parameter	Type	Description
field	String	The name of the field to select.

from(sObjectType)

Specifies the object to query. Returns an instance of Query.

This method accepts these parameters.

Parameter	Type	Description
sObjectType	String	The name of the object to query.

where(condition)

Specifies the where condition for the query. Returns an instance of Query.

This method accepts these parameters.

Parameter	Type	Description
condition	Condition	The condition to use as part of the query.

getRequiredFields()

Returns the required fields for a specified object type.

This method is supported only on desktop and doesn't work on the Life Sciences Cloud mobile app.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
    "Account",
    new SetCondition(
        "Id",
        "IN",
        new Query()
```

```
            .select("AccountId")
            .from("Visit")
            .where(new FieldCondition("Status", "=","Planned"))
    )
).build();

// Returns: "Id IN (SELECT AccountId FROM Visit WHERE Status = \'Planned\')"
```

SetCondition Class
SetCondition Class

Builds a SOQL condition for a field whose value is in a specified range. Extends OperatorCondition.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(field, operator, values)

Creates an instance of SetCondition.

This method accepts these parameters.

Parameter	Type	Description
field	String	The name of the field to use as part of a condition.
operator	String	The operator to use as part of a condition.
value	Array or Query	An array of values to use as part of a condition, or an instance of a Query object that specifies the field values to query.

getRequiredFields(objectType)

Returns the required fields for a specified object type. This method accepts these parameters.

Parameter	Type	Description
objectType	String	The name of the object to get the required fields for.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
   "Inquiry",
   new SetCondition(
       "Type",
       "IN",
       ["MedicalInquiry", "ProductInformation"],
   )
).build()

// Returns: "Type IN (\'MedicalInquiry\',\'ProductInformation\')"
```

Variables for Life Sciences Custom Scripts

Global variables in Life Sciences Cloud for Customer Engagement custom scripts provide access to contextual information such as environment details, database interactions, current record data, and user information. Use these variables to develop dynamic, data-driven custom scripts.

REQUIRED EDITIONS

Available in: Lightning Experience

Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Environment Variable

The environment (env) variable manages environment settings. You can retrieve or create environment option values, get log information, and format labels for localization.

Database Variable

The db object provides access to the database to query records. All db functions are executed synchronously on the Life Sciences Cloud mobile app and asynchronously on desktop. In order to query correctly, use await syntax.

Record Variable

The record variable represents the current record in the context of the custom script. You can retrieve string, number, boolean, or date field values, and you can set values for fields on the current record.

User Variable

The user variable represents the current user and provides methods to access and modify field values on the user record. You can retrieve string, number, boolean, or date field values, and you can set values for user record fields.

Environment Variable
Environment Variable

The environment (env) variable manages environment settings. You can retrieve or create environment option values, get log information, and format labels for localization.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

getOption(name)

Returns the value of the environment option. These options are set by default.

- **fromStatus**:
  - On desktop, fromStatus is set to the current value of a controlling field for the workflow path. This is set only when a Record Update action runs.
  - On the Life Sciences Cloud for Customer Engagement mobile app, fromStatus is set to the current value of the field to be updated by a Record Update action. This is set only when a Record Update action runs.
- **toStatus**: The new value of the field to be updated by a Record Update action. This is set only when a Record Update action runs.
- **fieldName**: The name of the field to be updated by a Record Update action. This is set only when a Record Update action runs.
- **actionName**: The name of the action on the action button.

Parameter	Type	Description
name	String	The name of the option to retrieve.

setOption(name, value)

Sets an environment option based on the name and value.

Parameter	Type	Description
name	String	The name of the option to set.
value	Object	The value of the option to set.

log(msg)

Logs a message to the browser console with the prefix "JS LOG: ".

Parameter	Type	Description
msg	String	The message to be logged to the browser console.

formatCustomLabel(key, defaultValue, argumentsList)

Translates a given custom label to match the current user's locale and replaces %@ symbols with given arguments. The resulting JavaScript object should be set to the title attribute in the returned message.

Parameter	Type	Description
key	String	The name of the custom label.
defaultValue	String	A default value to use when no custom label is found.
argumentsList	Array	An array of strings to use to replace placeholders in custom labels.

Database Variable

Database Variable

The db object provides access to the database to query records. All db functions are executed synchronously on the Life Sciences Cloud mobile app and asynchronously on desktop. In order to query correctly, use await syntax.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

rowById(entity, id, selectedFields)

Retrieves a record for a given object from the database by using the record ID.

Parameter	Type	Description
entity	String	The name of the object type to retrieve.
id	String	The ID of the record to retrieve.
selectedFields	Array	The array of fields to select. This parameter is supported only on desktop.

rowsByEntity(entity, selectedFields)

Retrieves records from the database for a given object.

Parameter	Type	Description
entity	String	The name of the object type to retrieve.
selectedFields	Array	The array of fields to select. This parameter is supported only on desktop.

query(entity, whereClause, selectedFields)

Retrieves records from the database for a given object based on a WHERE clause.

Parameter	Type	Description
entity	String	The name of the object type to

Parameter	Type	Description
		retrieve.
whereClause	String	The SOQL WHERE clause to use in the query. You can use ConditionBuilder to build the WHERE clause.
selectedFields	Array	The array of fields to select. This parameter is supported only on desktop.

bulkQuery(queries)

Returns a key-value container `[String: [JsDbObject]]` where the key is uniqueRequestKey and the value is an array of queried records.

Parameter	Type	Description
uniqueRequestKey	String	The unique key to use to identify the specific query result.
sObjectName	String	The name of the object type to retrieve.
condition	ConditionEnhancedBuilder	The ConditionEnhancedBuilder object to use for the query.
selectedFields	Array	The array of fields to select. This parameter is supported only on desktop.

Examples

This is a `query()` example.

```
await db.query(
    "Inquiry",
    await new ConditionBuilder(
        "Inquiry",
        new FieldCondition("Type", "=", "MedicalInquiry")
    ).build(),
```

```
    ["Id", "Name", "Status", "Type"]
)

/**
 * Returns arrays of Inquiry records of from the local database. The full quer
y is:
 * "SELECT Id, Name, Status, Type FROM Inquiry WHERE Type = \'MedicalInquir
y\'"
 */
```

This is a `bulkQuery()` example.

```
let result = await db.bulkQuery([
   {
       uniqueRequestKey: "inquiryRequest",
       sObjectName: "Inquiry",
       selectedFields: ["Id", "Name", "Status", "Type"],
       condition: new ConditionEnhancedBuilder(
           "Inquiry",
           new FieldCondition("Type", "=", "MedicalInquiry")
       ).build(),
   },
   {
       uniqueRequestKey: "accountRequest",
       sObjectName: "Account",
       selectedFields: ["Id", "Name"],
       condition: new ConditionEnhancedBuilder(
           "Account",
           new FieldCondition("Name", "=", "sForce")
       ).build(),
   },
]);

/**
 * These queries are executed:
 *
 * "SELECT Id, Name, Status, Type FROM Inquiry WHERE Type = \'MedicalInquir
y\'"
 *
 * "SELECT Id, Name FROM Account WHERE Name = \'sForce\'"
 *
 * The results are combined into a single key-value container, where the key i
s uniqueRequestKey and the value is an array of records. These can be treated
```

```
as JavaScript objects, and results can be accessed by properties.
 *
 * let inquiryRecords = result.inquiryRequest;
 * let accountRecords = result.accountRequest;
 */
```

Record Variable

Record Variable

The record variable represents the current record in the context of the custom script. You can retrieve string, number, boolean, or date field values, and you can set values for fields on the current record.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

stringValue(key)

Returns the value of a string field on the record if the field exists and is readable.

Parameter	Type	Description
key	String	The name of the field on the record.

numValue(key)

Returns the value of a number field on the record if the field exists and is readable.

Parameter	Type	Description
key	String	The name of the field on the record.

boolValue(key)

Returns the value of a boolean field on the record if the field exists and is readable.

Parameter	Type	Description
key	String	The name of the field on the record.

dateValue(key)

Returns the value of a date field on the record if the field exists and is readable.

Parameter	Type	Description
key	String	The name of the field on the record.

setValue(key, value)

Sets a value for a given field on a record.

Parameter	Type	Description
key	String	The name of the field on the record.
value	Object	The field value to set.

recordTypeName()

Returns the name of the current record's record type, if it exists.

User Variable
User Variable

The user variable represents the current user and provides methods to access and modify field values on the user record. You can retrieve string, number, boolean, or date field values, and you can set values for user record fields.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

stringValue(key)

Returns the value of a string field on the user record if the field exists and is readable.

Parameter	Type	Description
key	String	Name of the field on the user object.

numValue(key)

Returns the value of a number field on the user record if the field exists and is readable.

Parameter	Type	Description
key	String	Name of the field on the user object.

boolValue(key)

Returns the value of a boolean field on the user record if the field exists and is readable.

Parameter	Type	Description
key	String	Name of the field on the user object.

dateValue(key)

Returns the value of a date field on the user record if the field exists and is readable.

Parameter	Type	Description
key	String	The name of the field on the user object.

setValue(key, value)

Sets a value for a given field on a user record.

Parameter	Type	Description
key	String	The name of the field on the user object.

Parameter	Type	Description
value	Object	The field value to set.

Functions for Life Sciences Custom Scripts

Use functions to enhance the robustness and adaptability of your custom scripts in Life Sciences Cloud for Customer Engagement. These functions provide specific utilities, from error handling and namespace management to retrieving metadata about available fields.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

enableAccessErrors()

Enables returning exceptions in the custom script.

Set Up Data and Analytics

Integrate Life Sciences Cloud with Tableau Next to transform complex life sciences data into actionable insights and enhanced visualizations. Tableau Next requires Data 360 and the Data 360 semantic layer. Set up Sales Data reports to help your users better understand their customers and plan effective interactions.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Set Up Tableau Next for Life Sciences Cloud
Life Sciences Cloud integrates with Tableau Next to transform complex life sciences data into actionable insights and enhanced visualizations. Tableau Next is the composable, AI-analytics platform that turns any type of data into actionable insights. Tableau Next requires Data 360 and the Data 360 semantic layer.

Set Up a Tableau View Lighting Web Component for Your Life Sciences Org and Mobile App